

Decision tree learning

Sunita Sarawagi
IIT Bombay

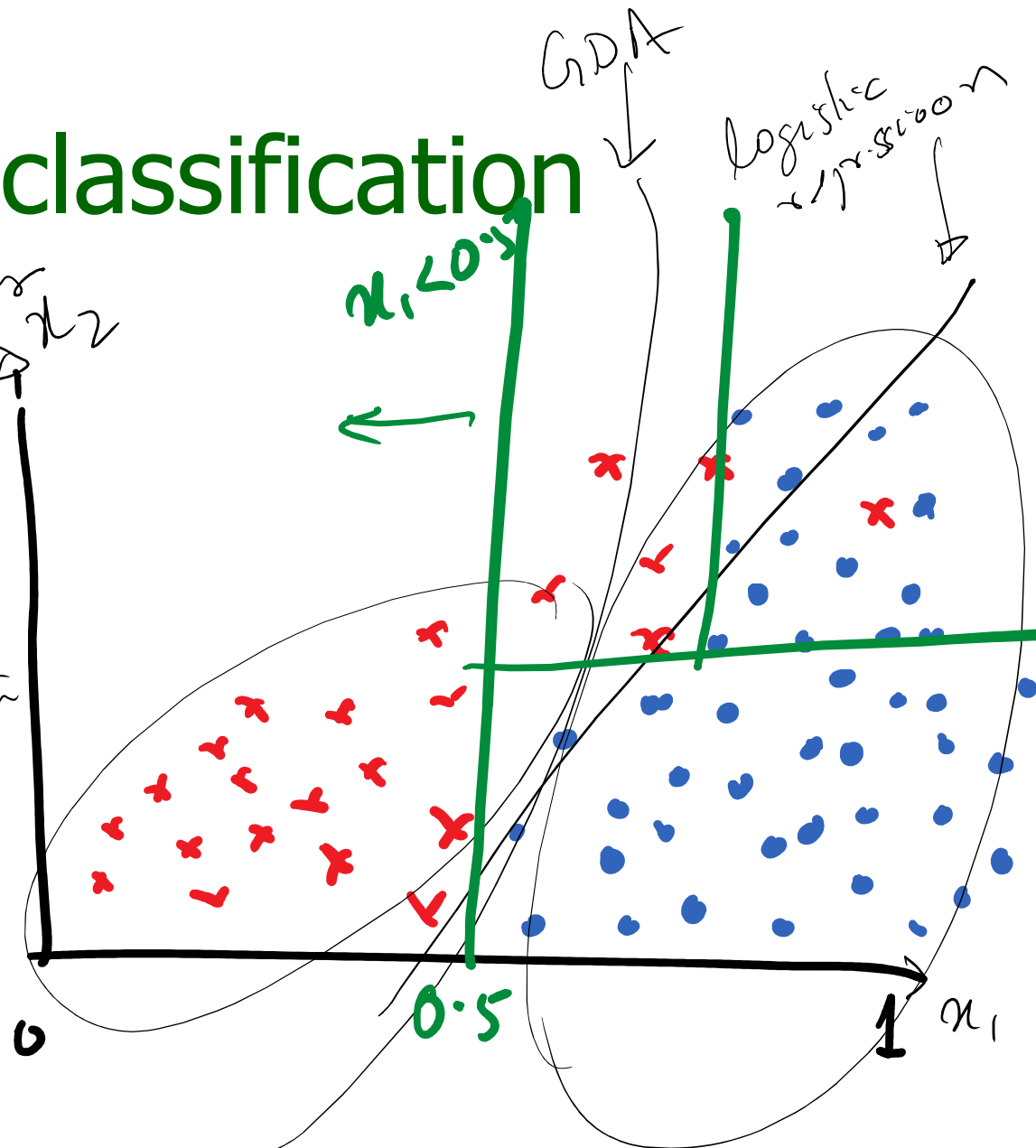
<http://www.cse.iitb.ac.in/~sunita>

Forms of classification

1) Conditional probabilistic classifier
+ Linear logistic regression.

2) Generative classifiers
eg: Gaussian Discriminant analysis (GDA)

3) Discrimination classifiers,
eg: decision trees.



$$x_1 < v_1$$

$$x_1 < v_3$$

$$x_2 < v_2$$

$$R$$

$$B$$

$$R$$

$$B$$

Decision tree classifiers

- Widely used learning method
 - Easy to interpret: can be re-represented as if-then-else rules
 - Approximates function by piece wise constant regions
 - Does not require any prior knowledge of data distribution, works well on noisy data.
 - Handles categorical data natively
 - Has been applied to:
 - classify medical patients based on the disease,
 - equipment malfunction by cause,
 - loan applicant by likelihood of payment.
 - lots and lots of other applications..
- 

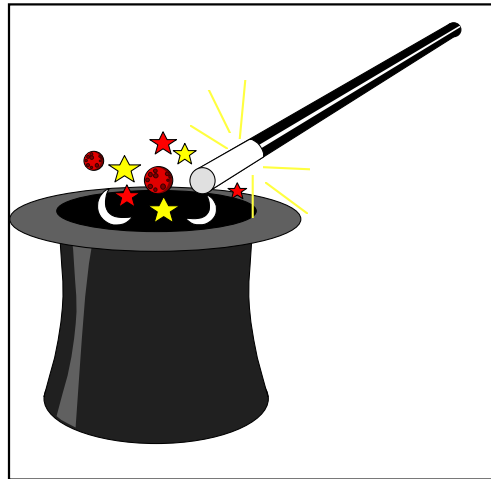
Setting

- Given old data about customers and payments, predict new applicant's loan eligibility.

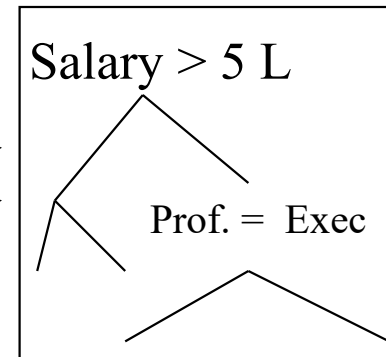
Previous customers

Age
Salary
Profession
Location
Customer type

Classifier



Decision rules

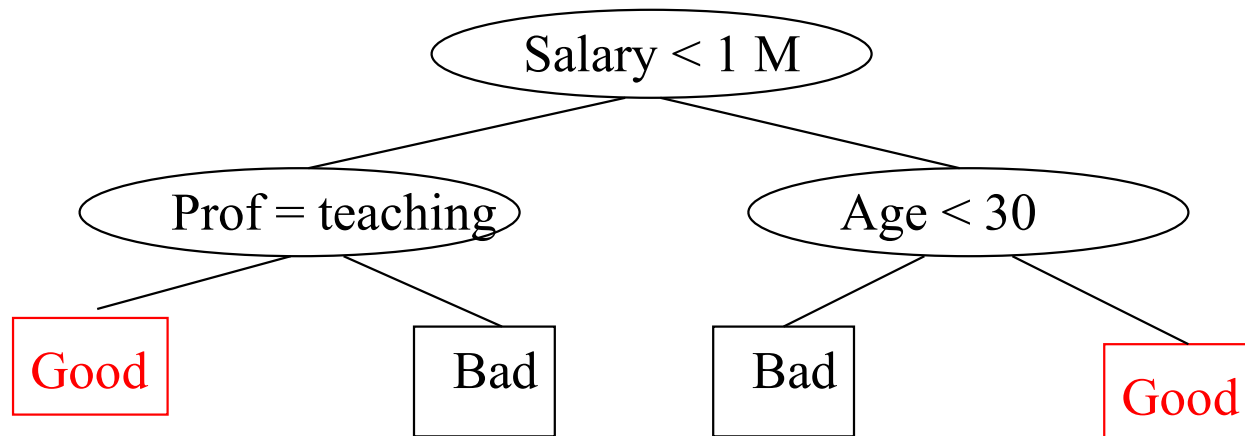


Good/
bad

New applicant's data

Decision trees

- Tree where internal nodes are simple decision rules on one or more attributes and leaf nodes are predicted class labels.

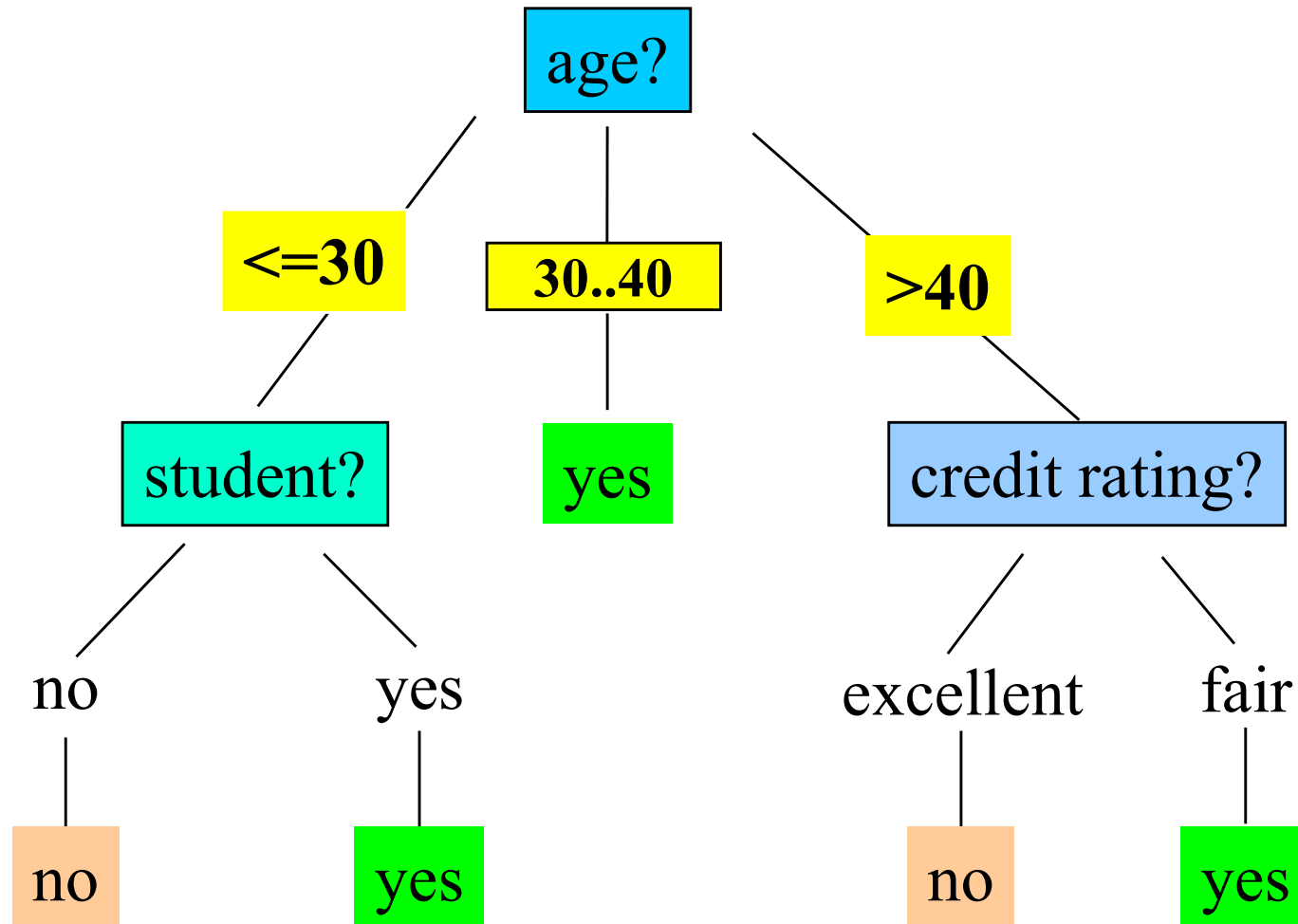


Training Dataset

This follows an example from Quinlan's ID3

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
30...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	yes
31...40	high	yes	fair	yes
>40	medium	no	excellent	no

Output: A Decision Tree for *"buys_computer"*

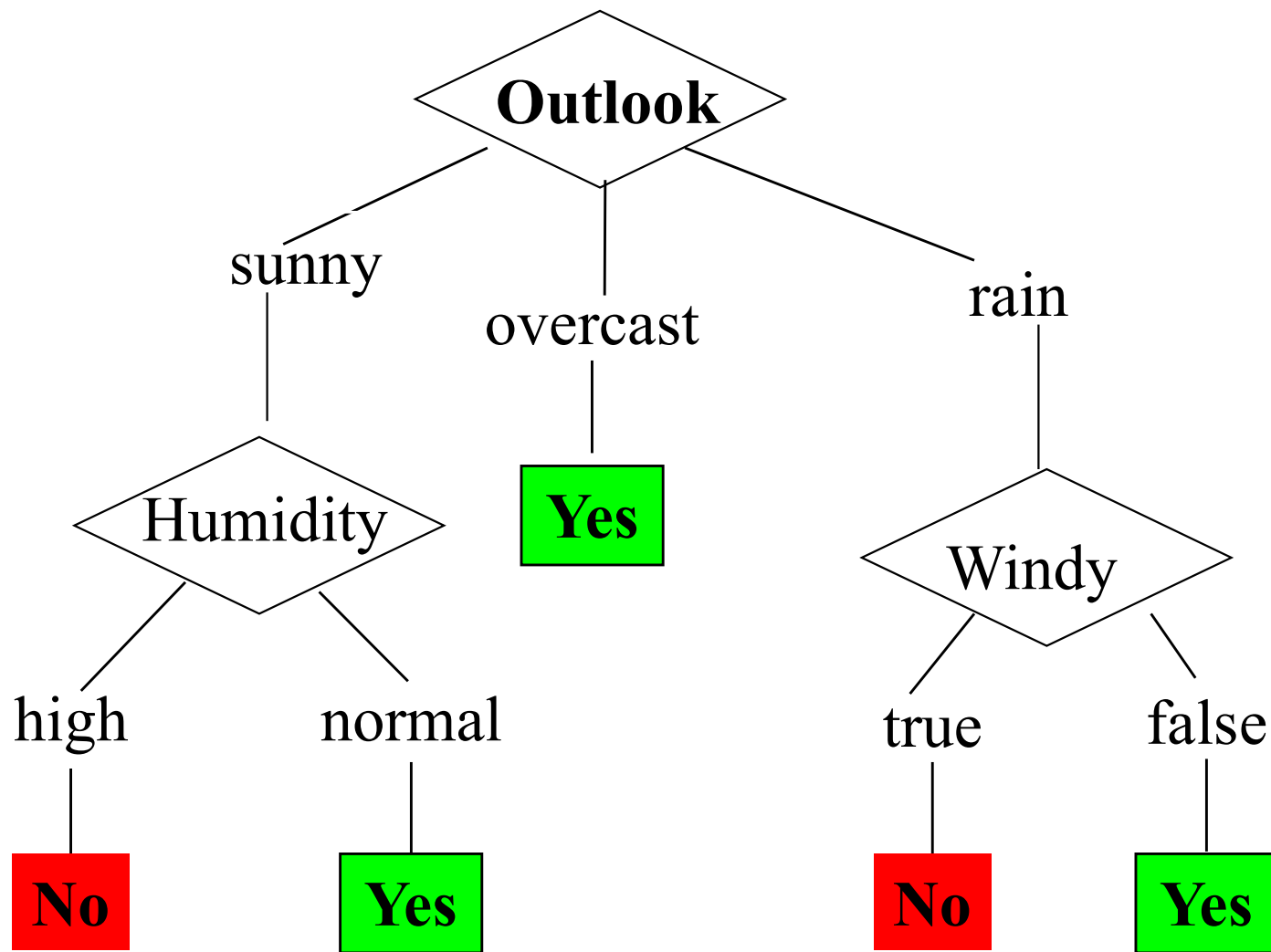


Weather Data: Play or not Play?

Outlook	Temperature	Humidity	Windy	Play?
sunny	hot	high	false	No
sunny	hot	high	true	No
overcast	hot	high	false	Yes
rain	mild	high	false	Yes
rain	cool	normal	false	Yes
rain	cool	normal	true	No
overcast	cool	normal	true	Yes
sunny	mild	high	false	No
sunny	cool	normal	false	Yes
rain	mild	normal	false	Yes
sunny	mild	normal	true	Yes
overcast	mild	high	true	Yes
overcast	hot	normal	false	Yes
rain	mild	high	true	No

*Note:
Outlook is the
Forecast,
no relation to
Microsoft
email program*

Example Tree for "Play?"



Topics to be covered

- Tree construction:
 - Basic tree learning algorithm
 - Measures of predictive ability
 - High performance decision tree construction: Sprint
- Tree pruning:
 - Why prune
 - Methods of pruning
- Other issues:
 - Handling missing data
 - Continuous class labels
 - Effect of training size

Tree learning algorithms

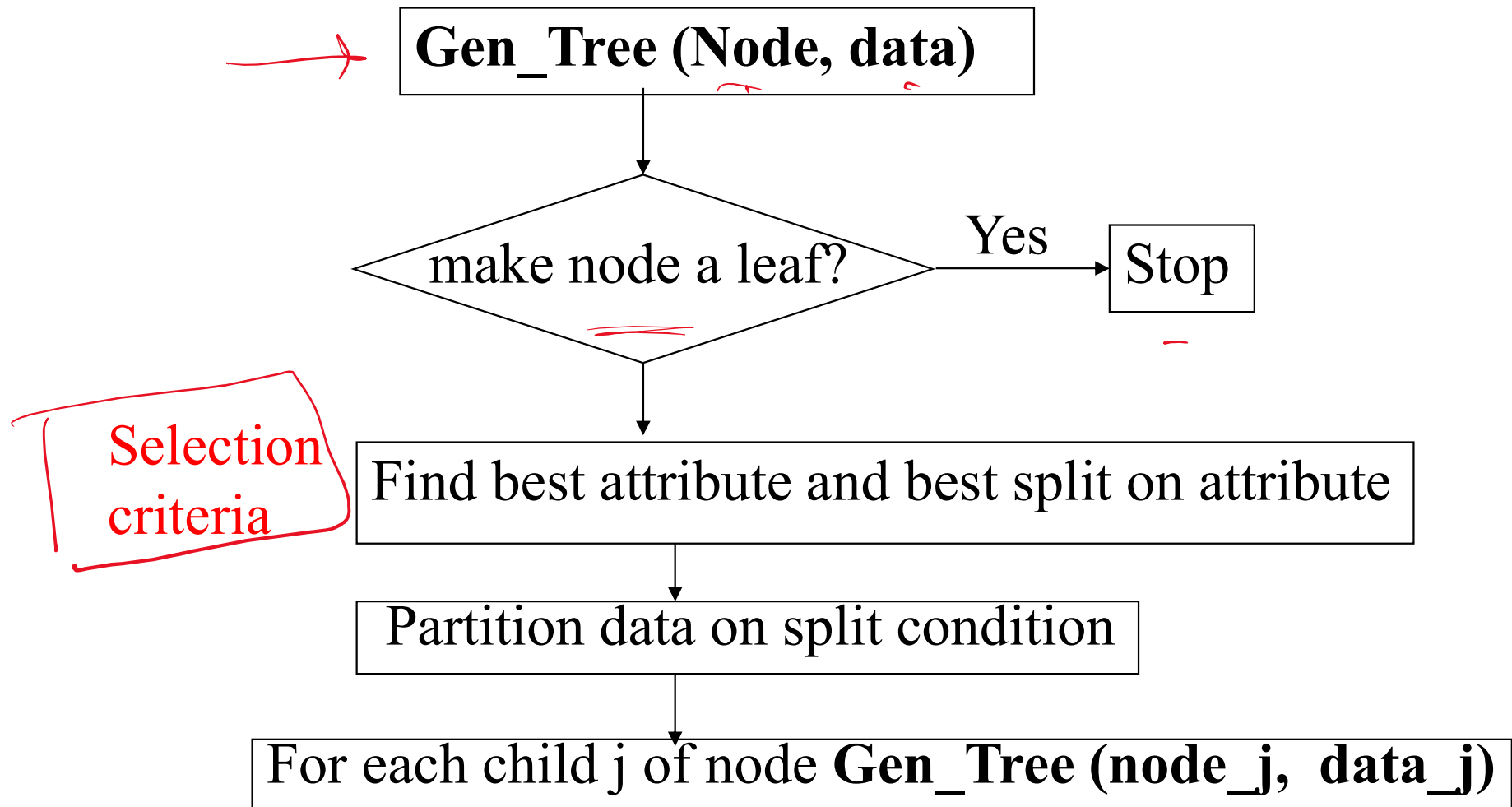
- ID3 (Quinlan 1986)
- Successor C4.5 (Quinlan 1993)
- CART
- SLIQ (Mehta et al)
- SPRINT (Shafer et al)

Tree learning algorithms

- Cannot use numerical optimization methods e.g. gradient descent algorithms since learning the tree structure poses a severely non-differentiable process

Basic algorithm for tree building

- Greedy top-down construction.



Split criteria

- Select the attribute that is best for classification.
- Intuitively pick one that best separates instances of different classes.
- Quantifying the intuitive: measuring separability:
- First define *impurity* of an arbitrary set S consisting of K classes
- Smallest when consisting of only one class, highest when all classes in equal number.
- Should allow computations in multiple stages.

classes = k

Measures of impurity

- Entropy

$$\text{Entropy } (\underline{S}) = - \sum_{i=1}^k p_i \log p_i$$

$p_1, p_2, \dots, p_k$
denote the
fraction of
examples in
classes

1, 2, $\dots$, k

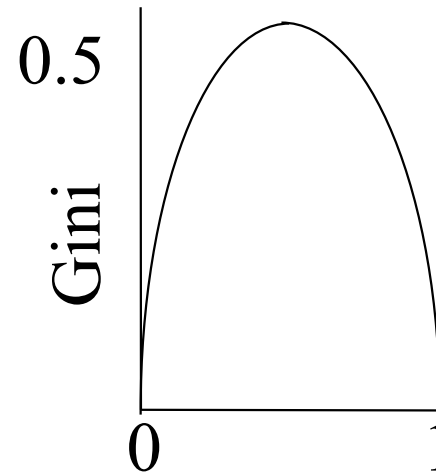
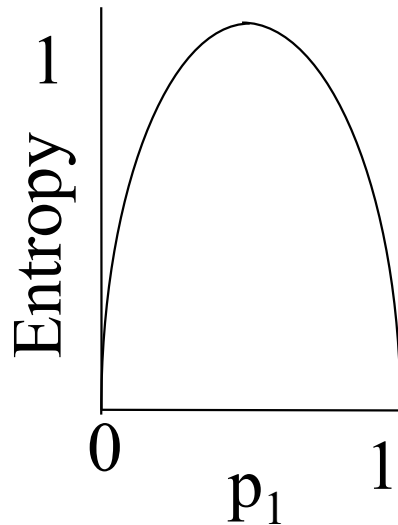
~~and~~
respectively
in set S .

- Gini

$$\text{Gini } (S) = 1 - \sum_{i=1}^k p_i^2$$

Information gain

$k = 2$



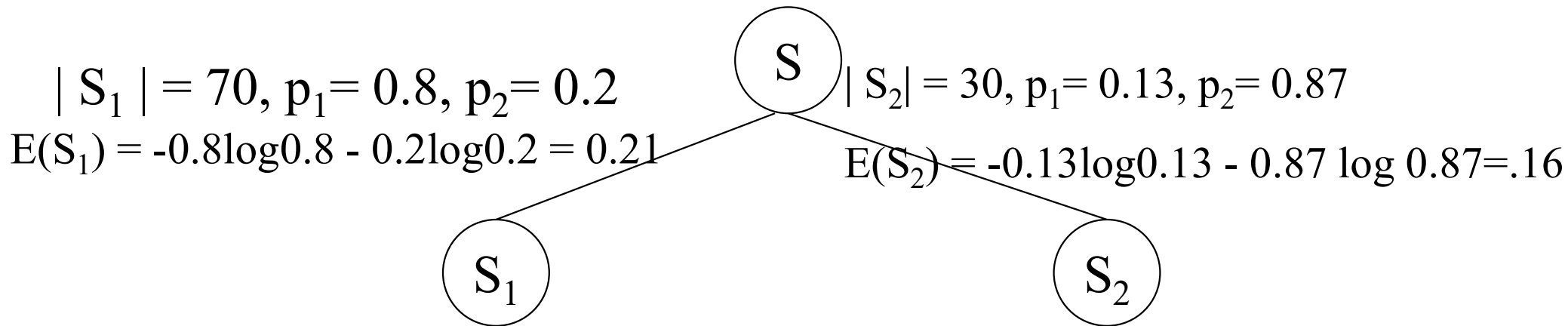
- Information gain on partitioning S into r subsets
- Impurity (S) - sum of weighted impurity of each subset

$$Gain(S, S_1..S_r) = Entropy(S) - \sum_{j=1}^r \left| \frac{S_j}{S} \right| Entropy(S_j)$$

fraction of examples in set S_j

Information gain: example

$$K=2, |S|=100, p_1=0.6, p_2=0.4$$
$$E(S) = -0.6 \log(0.6) - 0.4 \log(0.4) = 0.29$$



$$\text{Information gain: } E(S) - (0.7 E(S_1) + 0.3 E(S_2)) = 0.1$$

Weather Data: Play or not Play?

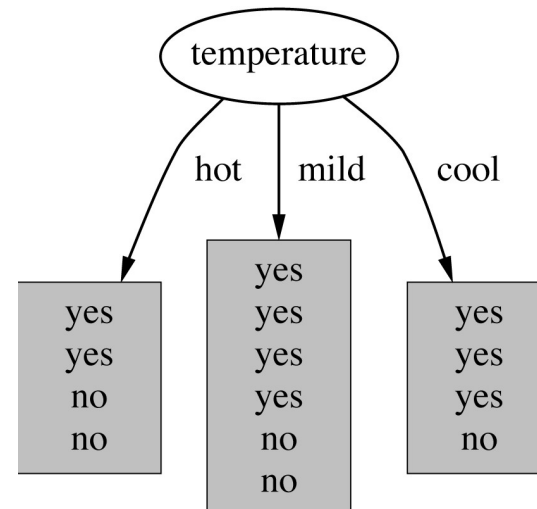
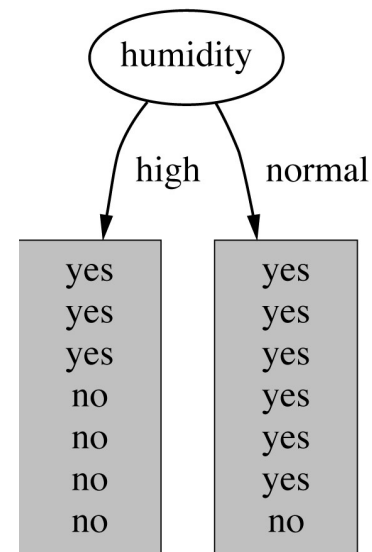
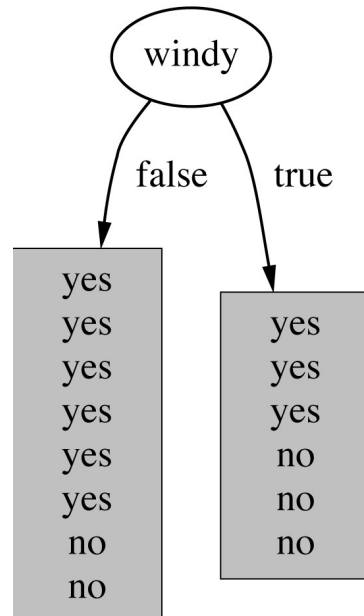
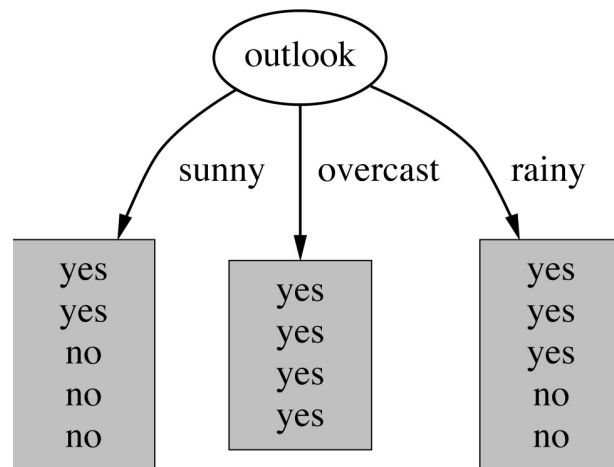
D
=

Outlook	Temperature	Humidity	Windy	Play?
sunny	hot	high	false	No
sunny	hot	high	true	No
overcast	hot	high	false	Yes
rain	mild	high	false	Yes
rain	cool	normal	false	Yes
rain	cool	normal	true	No
overcast	cool	normal	true	Yes
sunny	mild	high	false	No
sunny	cool	normal	false	Yes
rain	mild	normal	false	Yes
sunny	mild	normal	true	Yes
overcast	mild	high	true	Yes
overcast	hot	normal	false	Yes
rain	mild	high	true	No

root node



Which attribute to select?



Example: attribute "Outlook"

- "Outlook" = "Sunny":

$$\text{info}([2,3]) = \text{entropy}(2/5, 3/5) = -2/5 \log(2/5) - 3/5 \log(3/5) = 0.971 \text{ bits}$$

- "Outlook" = "Overcast":

$$\text{info}([4,0]) = \text{entropy}(1,0) = -1 \log(1) - 0 \log(0) = 0 \text{ bits}$$



*Note: $\log(0)$ is not defined, but we evaluate $0 * \log(0)$ as zero*

- "Outlook" = "Rainy":

$$\text{info}([3,2]) = \text{entropy}(3/5, 2/5) = -3/5 \log(3/5) - 2/5 \log(2/5) = 0.971 \text{ bits}$$

- Expected information for attribute:

$$\begin{aligned} \text{info}([3,2], [4,0], [3,2]) &= (5/14) \times 0.971 + (4/14) \times 0 + (5/14) \times 0.971 \\ &= 0.693 \text{ bits} \end{aligned}$$

Computing the information gain

- Information gain:
(information before split) – (information after split)

$$\text{gain("Outlook")} = \text{info}([9,5]) - \text{info}([2,3],[4,0],[3,2]) = 0.940 - 0.693 \\ = 0.247 \text{ bits}$$

- Information gain for attributes from weather data:

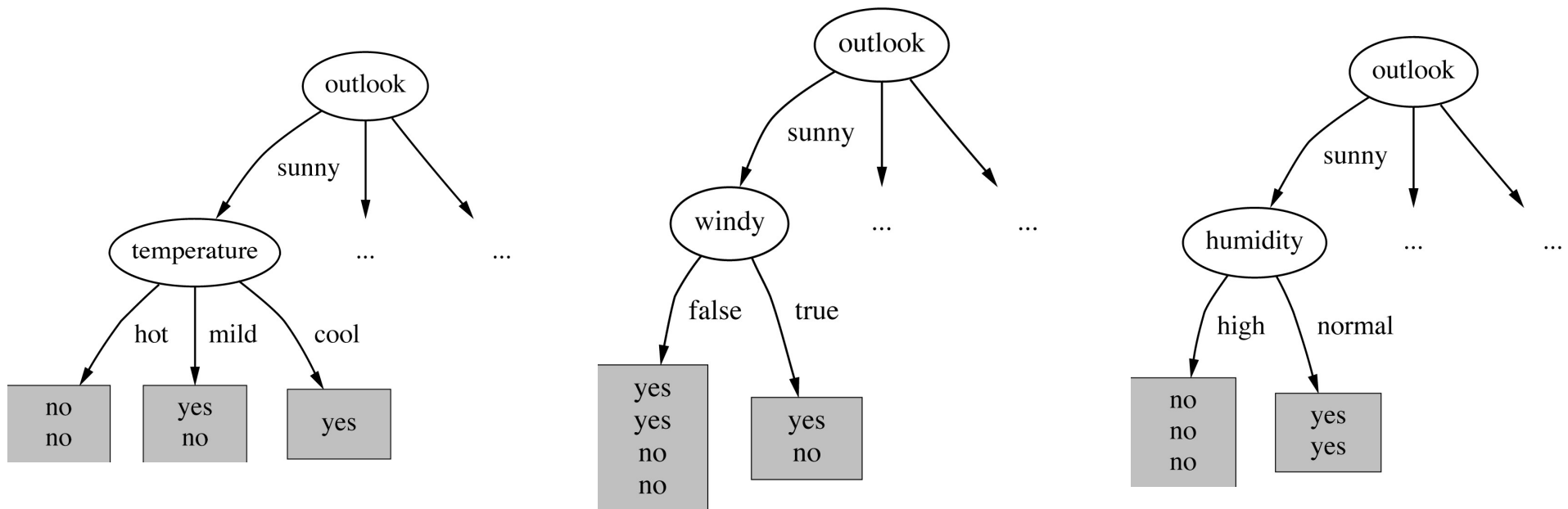
$$\text{gain("Outlook")} = 0.247 \text{ bits}$$

$$\text{gain("Temperature")} = 0.029 \text{ bits}$$

$$\text{gain("Humidity")} = 0.152 \text{ bits}$$

$$\text{gain("Windy")} = 0.048 \text{ bits}$$

Continuing to split

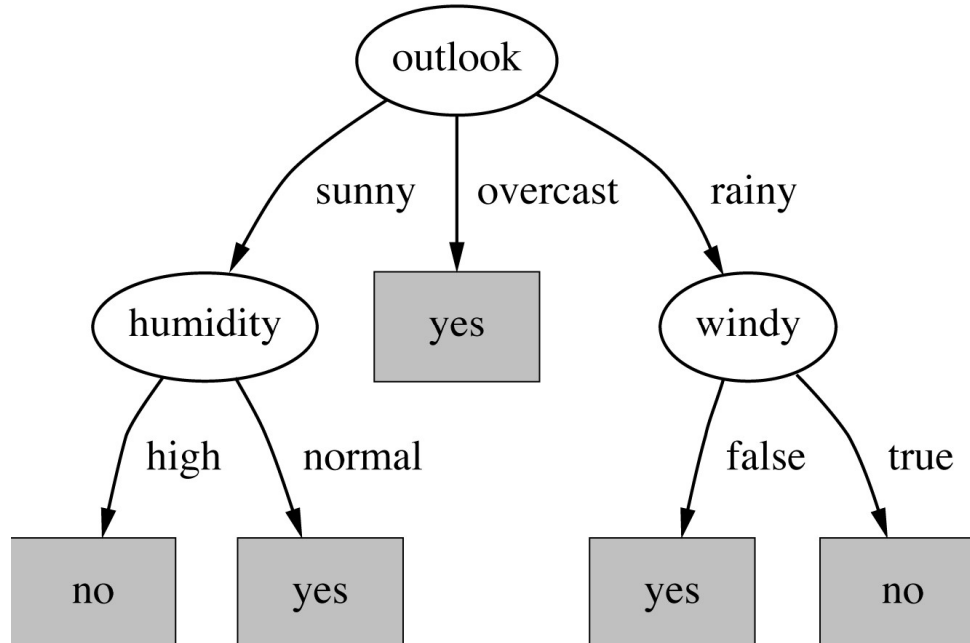


$\text{gain}(\text{"Temperature"}) = 0.571 \text{ bits}$

$\text{gain}(\text{"Humidity"}) = 0.971 \text{ bits}$

$\text{gain}(\text{"Windy"}) = 0.020 \text{ bits}$

The final decision tree

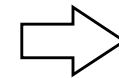
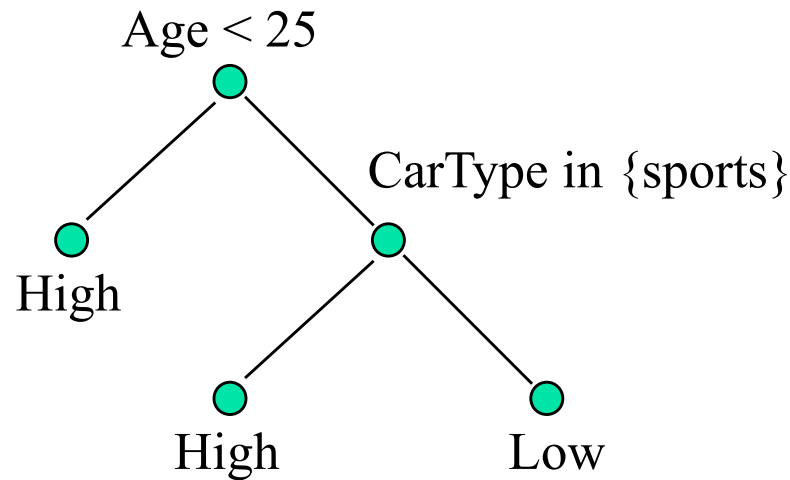


- Note: not all leaves need to be pure; sometimes identical instances have different classes
⇒ Splitting stops when data can't be split any further

Splitting on continuous attributes

- Example Data

Age	Car Type
42	family
17	truck
57	sports
21	sports
28	family
68	truck



<i>Risk</i>
<i>Low</i>
<i>High</i>
<i>High</i>
<i>High</i>
<i>Low</i>
<i>Low</i>

Splitting on continuous attributes

- Consider splits of form: $value(A) < x$
 - Example: Age < 17
- Evaluating this split-form for every value in an attribute list
 - For each split threshold, we need to compute info-gain and choose the best.
 - Exhaustive enumeration of all possible thresholds expensive.

An efficient algorithm for finding the best split on a continuous column

- We only need to consider split-thresholds which appear in the training data
- If we order the data on that column, then we can incrementally compute the info-gain of successive values.

To evaluate splits on attribute A for a given tree-node:

Initialize class-histogram of left child to zeroes;
Initialize class-histogram of right child to same as its parent;
Sort the data in increasing order of column A .

for each record in the attribute list **do**
 evaluate splitting index for $value(A) < record.value$;
 using class label of the record, update class histograms;

Split Points: Continuous Attrib.

Attribute List

Age	Risk
17	High
20	High
23	High
32	Low
43	High
68	Low

Position of
cursor in scan

← 0: Age < 17 →
 ← 1: Age < 20 →
 ← 3: Age < 32 →
 ← 6 →

State of Class Histograms:

Left Child

Right Child

High	Low
0	0

High	Low
4	2

High	Low
1	0

High	Low
3	2

High	Low
3	0

High	Low
1	2

High	Low
4	2

High	Low
0	0

GINI Index:

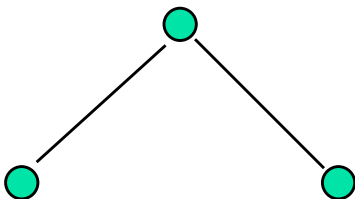
GINI = undef

GINI = 0.4

GINI = 0.222

GINI = undef

High	Low
4	2



Preventing overfitting

- A tree T overfits if there is another tree T' that gives higher error on the training data yet gives lower error on unseen data.
- An overfitted tree does not generalize to unseen instances.
- Happens when data contains noise or irrelevant attributes and training size is small.
- Overfitting can reduce accuracy drastically:
 - 10-25% as reported in Minger's 1989 Machine learning
- Example of over-fitting with binary data.

Training Data Vs. Test Data Error Rates

- Compare error rates measured by
 - learn data
 - large test set
- Learn $R(T)$ always decreases as tree grows (Q: Why?)
- Test $R(T)$ first declines then increases (Q: Why?)
- Overfitting is the result tree of too much reliance on learn $R(T)$
- Can lead to disasters when applied to new data

No. Terminal Nodes	<u>R(T)</u>	<u>R^{ts}(T)</u>
71	.00	.42
63	.00	.40
58	.03	.39
40	.10	.32
34	.12	.32
19	.20	.31
**10	.29	.30
9	.32	.34
7	.41	.47
6	.46	.54
5	.53	.61
2	.75	.82
1	.86	.91

Digit recognition dataset: CART book

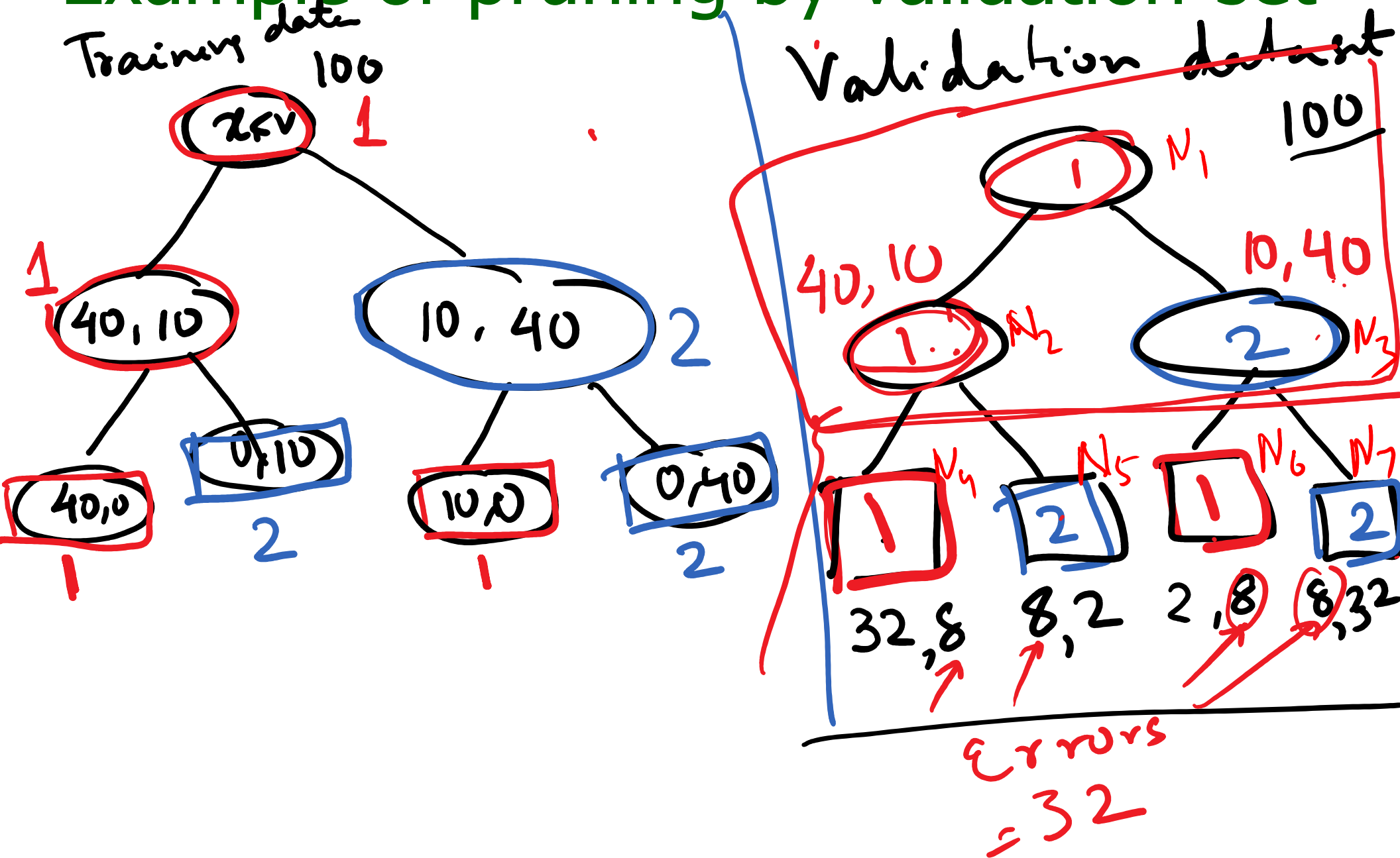
Approaches to prevent overfitting

- Two Approaches:
 - Stop growing the tree beyond a certain point
 - Tricky, since even when information gain is zero an attribute might be useful (XOR example)
 - First over-fit, then post prune. (More widely used)
 - Tree building divided into phases:
 - Growth phase
 - Prune phase

Cross validation

- Partition the dataset into two disjoint parts:
 - 1. Training set used for building the tree.
 - 2. Validation set used for pruning the tree:
 - Rule of thumb: 2/3rds training, 1/3rd validation
- Evaluate the tree on the validation set and prune bottom-up the tree for smallest error on validation set.
 - Each internal node is also assigned a label corresponding to the majority of examples at that.
 - At each leaf and internal node keep count of correctly labeled data.
 - Starting bottom-up, prune branches below a node with error less than its children.

Example of pruning by validation-set

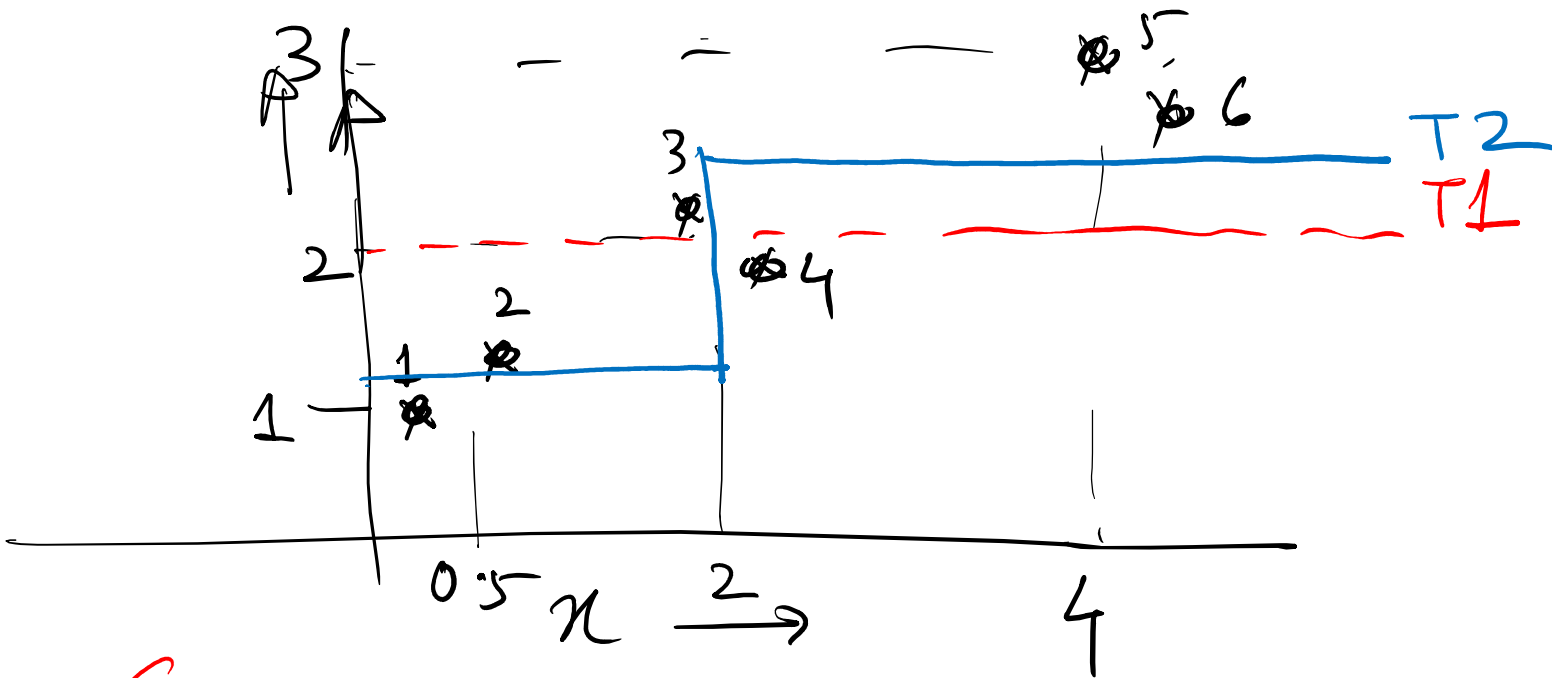


Overfitting example

- Consider the case where a single attribute x_j is adequate for classification but with an error of 20%
- Consider lots of other noise attributes that enable zero error during training
- This detailed tree during testing will have an expected error of $(0.8*0.2 + 0.2*0.8) = 32\%$ whereas the pruned tree with only a single split on x_j will have an error of only 20%.

Regression trees

- Decision tree with continuous class labels:
- Regression trees approximates the function with piece-wise constant regions.
- Split criteria for regression trees:
 - Predicted value for a set S = average of all values in S
 - Error: sum of the square of error of each member of S from the predicted average.
 - Pick smallest average error.
- Splits on categorical attributes:
 - Can it be better than for discrete class labels?
 - Homework.



S

$$D \equiv \{ (0.4, \underline{1}), (0.5, \underline{1.2}), (2.1, \underline{2}), (2.2, \underline{1.9}), (4, \underline{3}), (4.2, \underline{2.8}) \}$$

single-node tree!

T1

$$\frac{\sum_{i=1}^n y_i}{n}$$

$$\left\{ \begin{pmatrix} x^1 \\ y_1 \end{pmatrix}, \begin{pmatrix} x^2 \\ y_2 \end{pmatrix} \right\}$$

2-node tree

T2

$$n < 2$$

S1

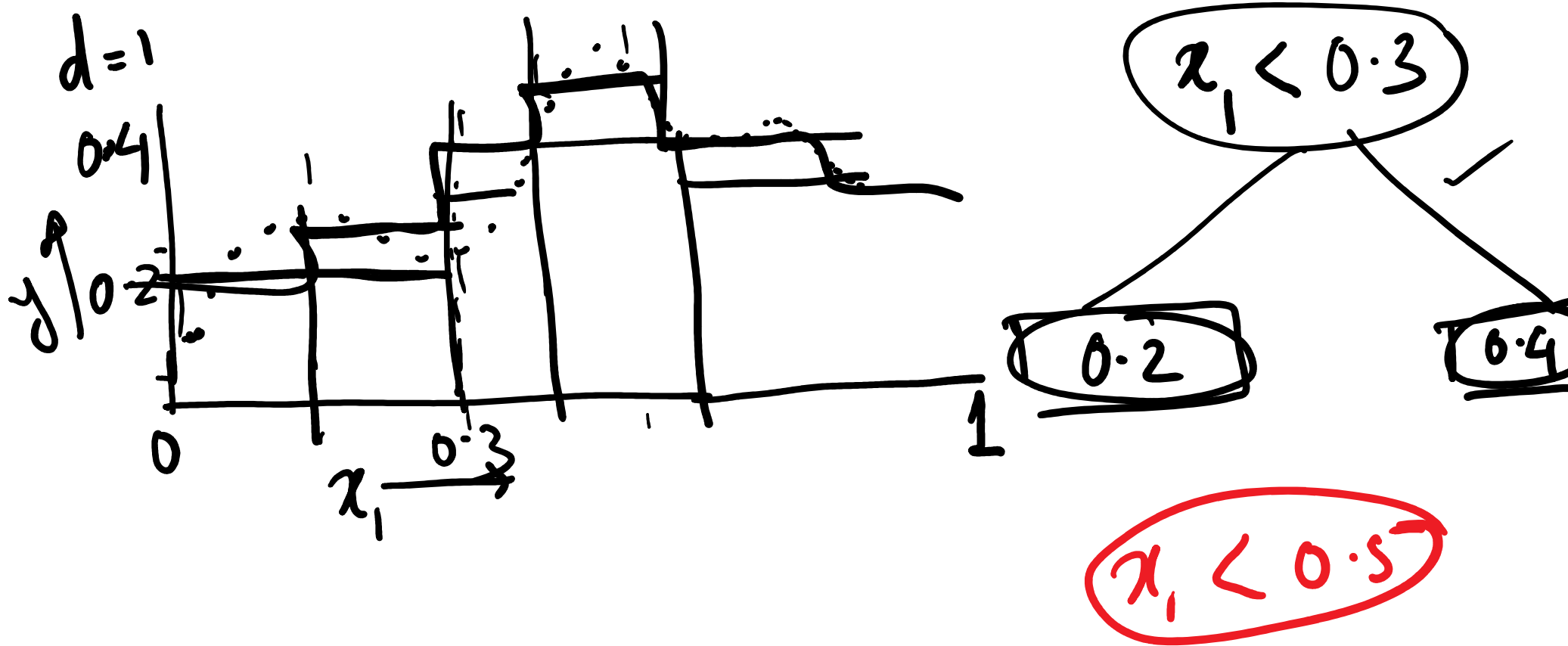
$$1.1$$

$$S_2 \equiv \{$$

$$2.4$$

$$= \frac{2 + 1.9 + 3 + 2.8}{4}$$

Regression tree example (1D)



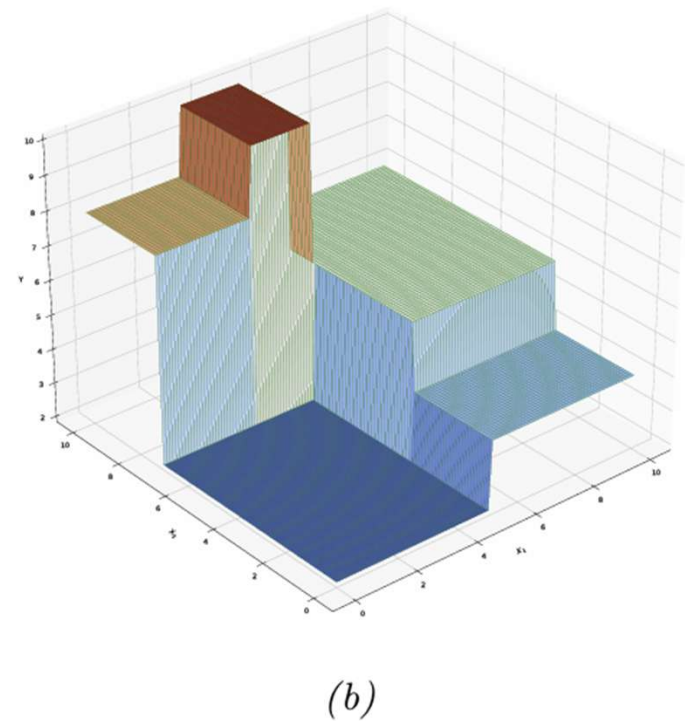
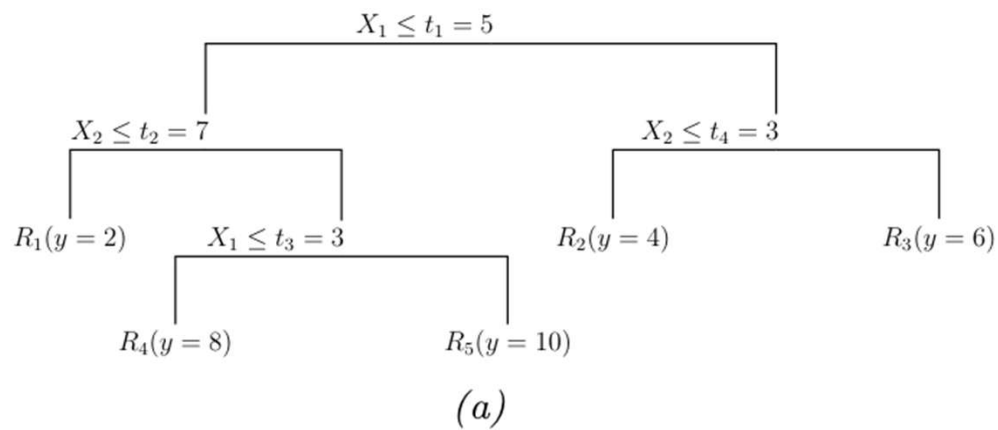


Figure 18.1: (a) A regression tree on two inputs. (b) Corresponding piecewise constant surface, where the regions have heights 2, 4, 6, 8 and 10. Adapted from Figure 9.2 of [HTF09]. Generated by [regtreeSurfaceDemo.ipynb](#).

Pros and Cons of decision trees

- Pros

- + Reasonable training time
- + Fast application
- + Easy to interpret
- + Easy to implement
- + Intuitive

- Cons

- Not effective for very high dimensional data where information about the class is spread in small ways over many correlated features
 - Example: words in text classification
- Not robust to dropping of important features even when correlated substitutes exist in data